

OpenPGP Email Encryption

Workshop



by Alex Chesnakov

About me

- Alex Chesnakov 
- Senior Engineer, Security Manager @ Health Gorilla
- Security
- Linux, AWS, CTFs
- You may see me sailing or flying kites at Shoreline Lake

All opinions are my own and not those of my employer.

Goals

- **Introduction to OpenPGP encryption**
- **Create OpenPGP keys**
- **Setup OpenPGP encryption in Thunderbird email client**
- **Exchange encrypted email messages with 2-3 recipients**
- **Encrypt, Decrypt, Sign and check Digital Signatures**

Encryption

Encryption Algorithms:

- Symmetric (AES256, encrypted zip file)
- Asymmetric (OpenPGP)

Usage:

- In transit (<https://>, WIFI encryption)
- At Rest (Mac and Windows disk encryption)
- End-to-End Encryption (OpenPGP)

Why

- [Before 1996 - Illegal to export](#)
 - [Encryption ban proposal in the UK](#)
 - [Encryption ban proposal in EU](#)
 - [Encryption ban proposal in the US](#)
-
- I want to encrypt a few files
 - No encryption necessary, I just want to be sure email indeed comes from the person
 - I found a 0-day but company insists I use their OpenPGP to encrypt email before sending
 - I downloaded .iso or program, they suggest to check signature, but I don't know how



Terminology

- [PGP](#) - Pretty Good Privacy Software. Proprietary, PGP Corp -> Symantec -> Broadcom
- [GPG](#) - GNU Privacy Guard Software. Free and open-source
- [OpenPGP](#) – Standard. PGP and GPG are OpenPGP-compliant

- Plaintext - “*Secret Message*” or binary file, like photo
- Ciphertext - “*Frperg Zrffntr*” encrypted text
- Cipher - algorithm for performing encryption/decryption. ROT13: A-N, B-O, C-P
- Hash - function map data of arbitrary size to fixed-size value output
 - Hash always has the same length for same hash algorithm (MD5, SHA256)
 - Change one bit in input, 50% different bits in hash
 - The same input always gives the same hash
 - One way function - cannot recover plaintext from a hash

Goals of PGP Cryptography

- **Confidentiality** - message contents remain private
- **Integrity** - refers to keeping a message unchanged
- **Nonrepudiation** - person cannot deny signing a particular message
- **Authenticity** - All these effects occurring simultaneously.

Main Usage Examples

Goal	How
I want anyone who reads this message to be sure I sent it and it was not changed (Nonrepudiation, Integrity)	Digitally sign the message with your private key
I want to verify the sender's identity	Verify signature with the sender's public key.
I want to send a message that only my intended recipient can read (Confidentiality)	Encrypt the message with the recipient's public key
I want to decrypt a message that I received	Decrypt the message with your private key
I want my message to be readable only by recipient, and I want recipient to be sure message came from me (Authenticity)	Encrypt the message with the recipient's public key and digitally sign the message with your private key
I want to decrypt and verify a message that includes a digital signature	Decrypt the message with your private key and verify the signature with the sender's public key

Confidentiality

Confidentiality means that the message contents remain private. The *plaintext* cannot be viewed by anyone who doesn't have the necessary keys.

In many cases, you cannot prevent someone from viewing the *ciphertext*, because every message over the Internet can be viewed by a large number of people, just as postcards can be viewed by a postman or a neighbor.



Integrity

Integrity refers to keeping a message unchanged. By using OpenPGP, you can confirm that a message has not been tampered with during transmission.

Kali Linux 2024.2 Changelog ⁸

64-bit 32-bit Apple Silicon (ARM64)

SHA256sum

```
5eb9dc96cccbdfef7
610d3cbced1bd6ee
89b5acdfc83ffef1
f06e6d02b058390c
```

↓ 4.0G torrent **sum**

 **64**

Weekly

Untested images with the latest updates

↓ 4.0G repository sum

 **64**

NetInstaller

All packages are downloaded during installation

↓ 495M torrent sum

 **64**

Everything

Image for air-gapped networks

torrent 11G sum

Nonrepudiation

Nonrepudiation means that a person cannot deny signing a particular message, which is especially important in the context of email.

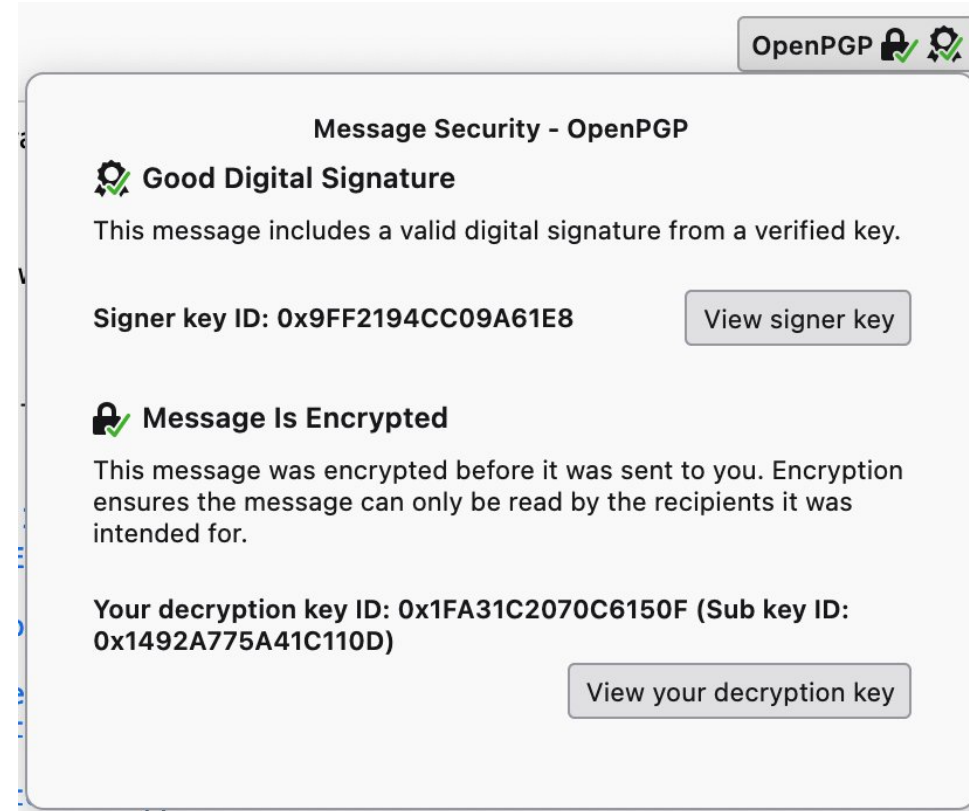
Example of nonrepudiation:

- Software vendors **Sign** program, drivers, .iso files
- DocuSign

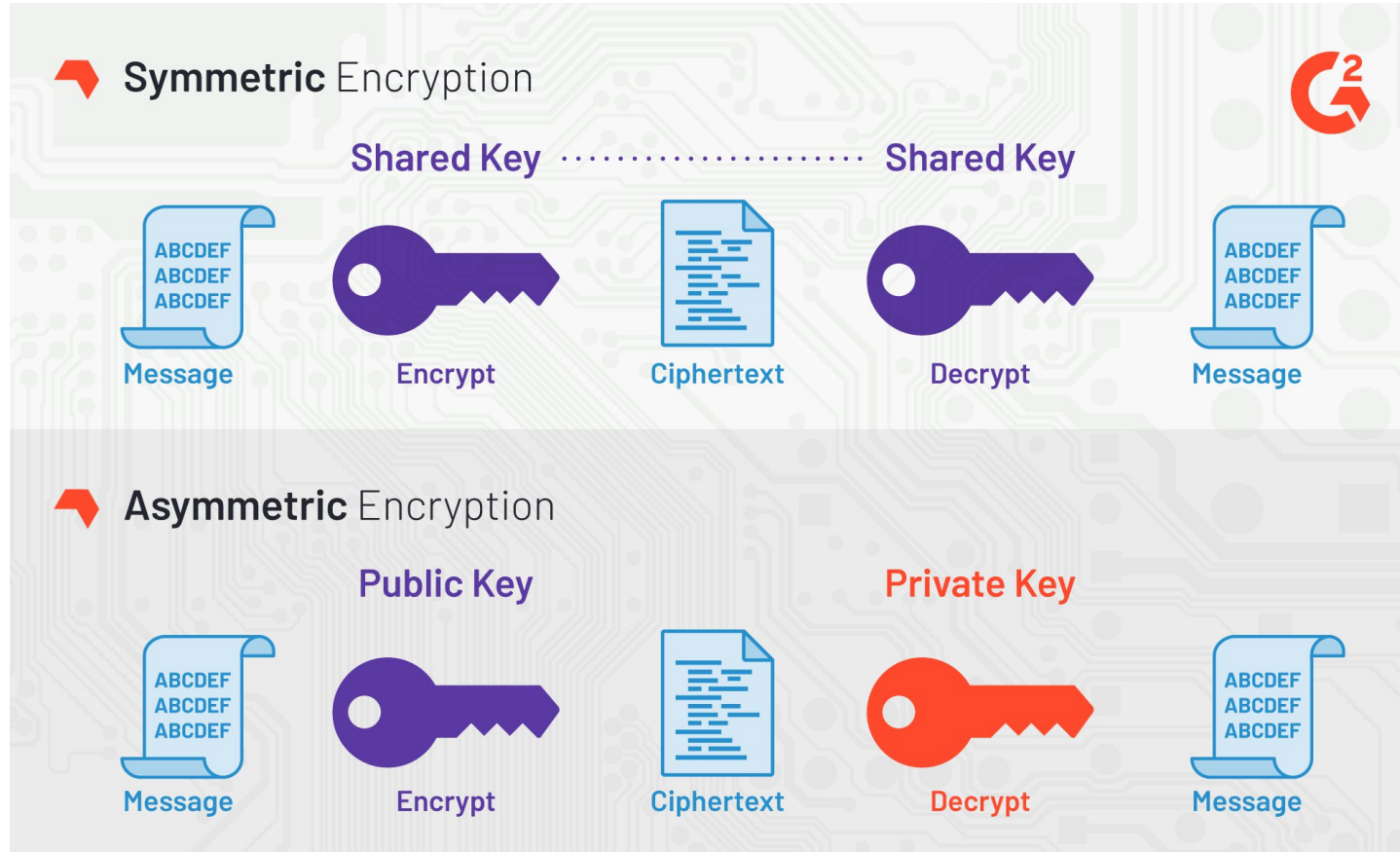


Authenticity

When you receive an email that has been **encrypted** and **signed** with OpenPGP, you know that the contents of the message have been **concealed** from any eavesdroppers. You know that the content of the message has **not** been **changed**. You also know that the message **comes** from the **person** who has the right to send such a message in the sender's name



Symmetric vs Asymmetric Encryption



Asymmetric Cryptography

- **Pair of keys**

Encrypt with one key, decrypt with another key.

You can encrypt with **public** key and decrypt with **private** key

You can encrypt with **private** key and decrypt with **public** key

- **Public key** – share the key with anyone – [Public-Key Cryptography](#)
- **Private key** – do not share, keep private
- **Keyserver** – server that provides directory of OpenPGP keys

Thunderbird – easy way to start with OpenPGP

Thunderbird - <https://www.thunderbird.net/>

Mac:

```
brew install --cask thunderbird
```

Kali Linux (Thunderbird is probably installed already):

```
sudo apt update
```

```
sudo apt install thunderbird
```

Windows:

<https://www.thunderbird.net/> Click Download. Install

Thunderbird: Add Account

Sign in with your existing email

NOTE: For Google and Outlook
do not enter password, Click Continue

The screenshot shows the 'Account Setup' window in Thunderbird. The title bar includes a folder icon, the text 'Account Setup', and a close button. Below the title bar, the main heading is 'Set Up Your Existing Email Address'. Underneath, a message states: 'To use your current email address fill in your credentials. Thunderbird will automatically search for a working and recommended server configuration.' There are three input fields: 'Your full name' with the value 'Alex C', 'Email address' with the value 'john.doe@example.com', and 'Password' which is empty. The 'Email address' field is highlighted with a red underline. To the right of the 'Email address' field is a link that says 'Get a new email address', which is also underlined in red. Below the input fields is a checkbox labeled 'Remember password' which is checked. At the bottom right are two buttons: 'Cancel' and 'Continue'. The 'Continue' button is highlighted with a red underline. At the very bottom, a small note says 'Your credentials will only be stored locally on your computer.'

Account Setup

Set Up Your Existing Email Address

To use your current email address fill in your credentials.
Thunderbird will automatically search for a working and recommended server configuration.

Your full name

Alex C

Email address

john.doe@example.com

[Get a new email address](#)

Password

☒ Remember password

Cancel Continue

Your credentials will only be stored locally on your computer.

Thunderbird: Generate new Key Pair

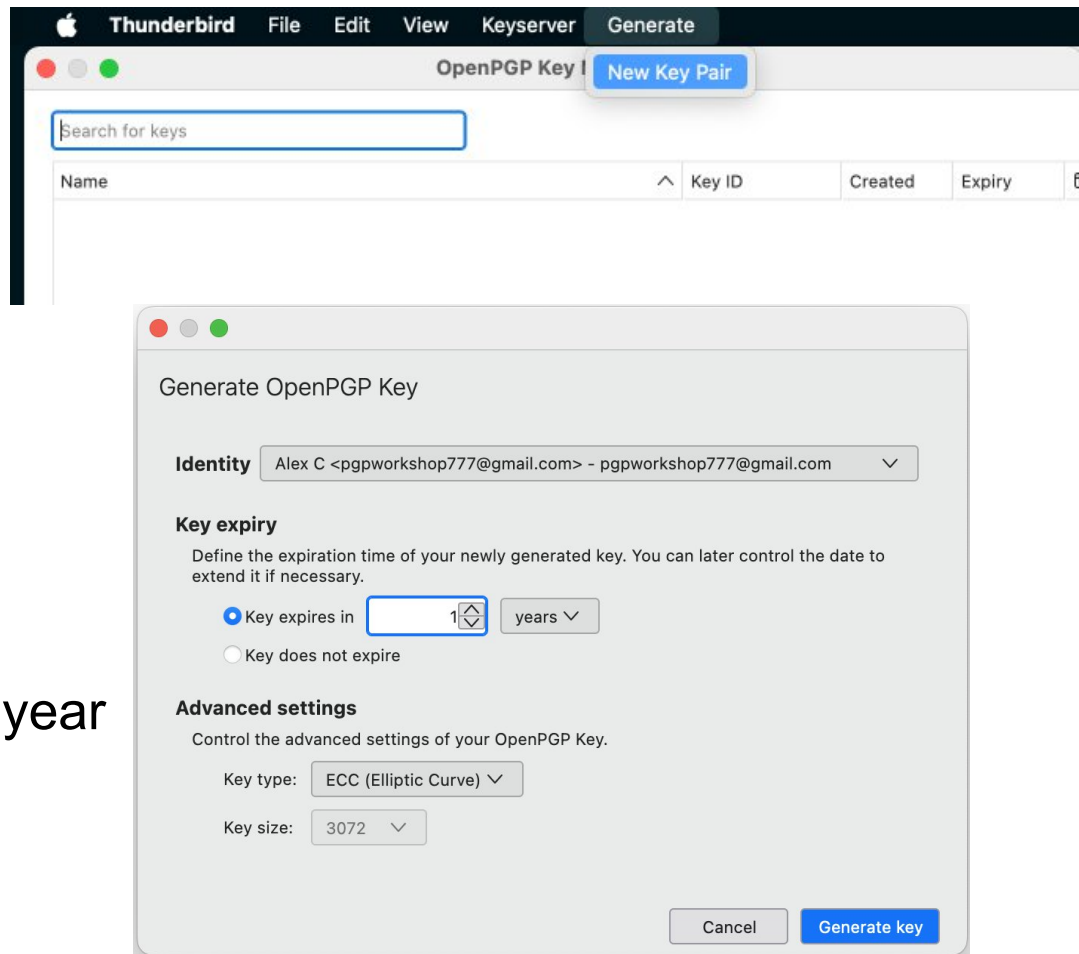
In Thunderbird:

Tools -> OpenPGP Key Manager

Generate -> New Key Pair

Defaults will work

You may Expire your first keys in 1 year
and use ECC Key (much shorter)



Thunderbird: use the OpenPGP Keys

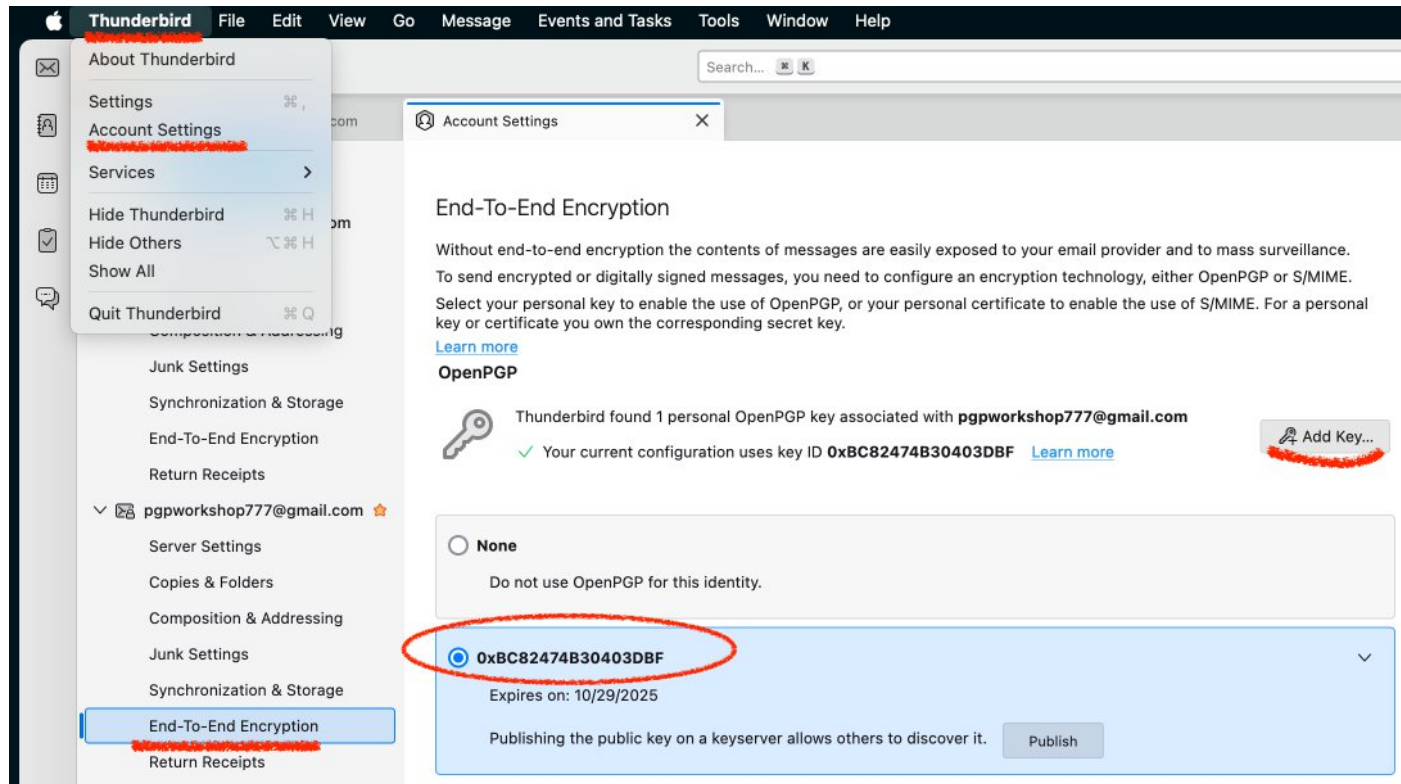
In Thunderbird:

Thunderbird ->

Account Settings ->

End-To-End Encryption ->
Choose your Keys

May need to Add Key
(if keys do not shows up)

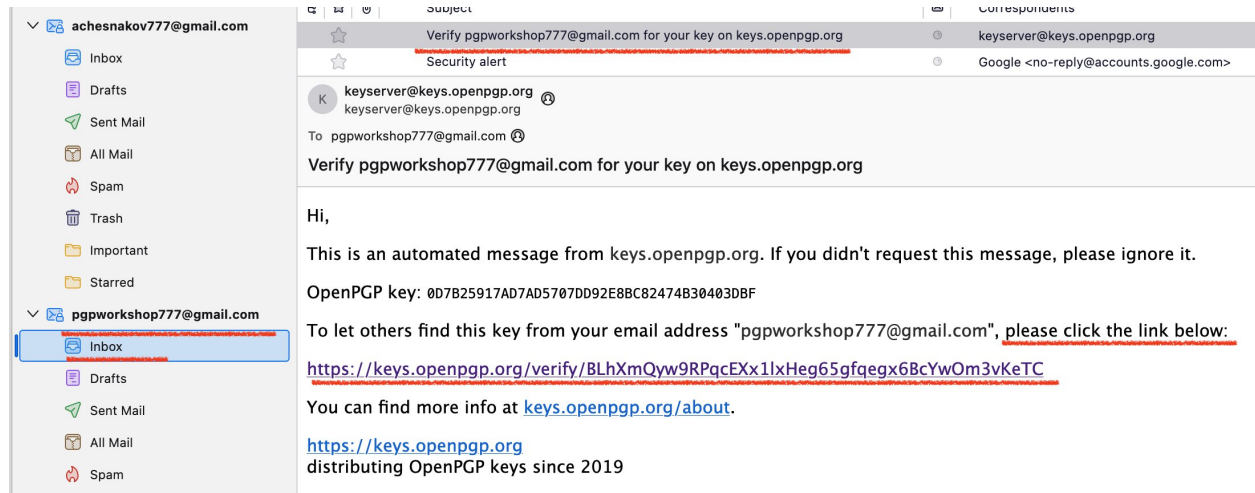
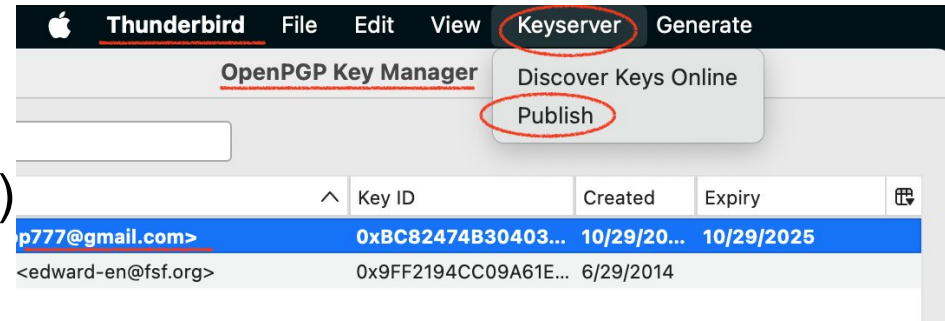


Publish Keys (Optional)

You may Publish the key
(Tools -> OpenPGP Key Manager->Keyserver)
This will help others find your OpenPGP keys
but gives away info about you (privacy)

You should receive an email and
confirm that you the owner of
this email account

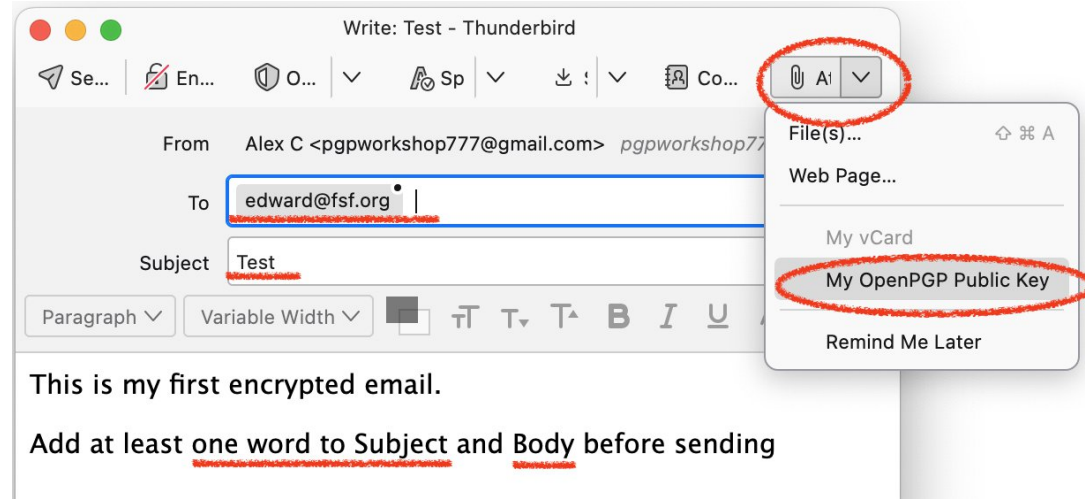
This will make the key
discoverable through the search
at <https://keys.openpgp.org/>



Email Edward your PubKey

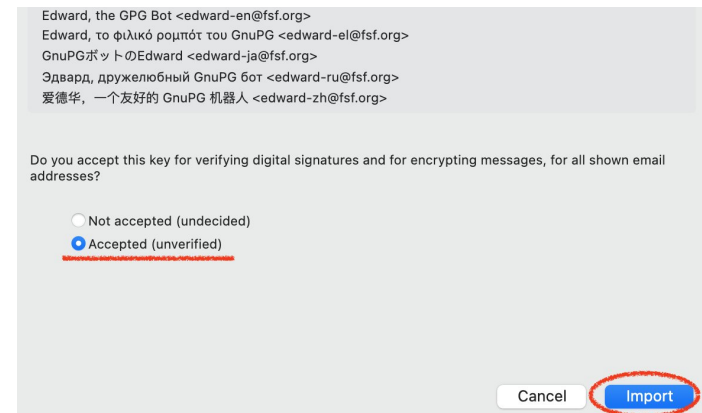
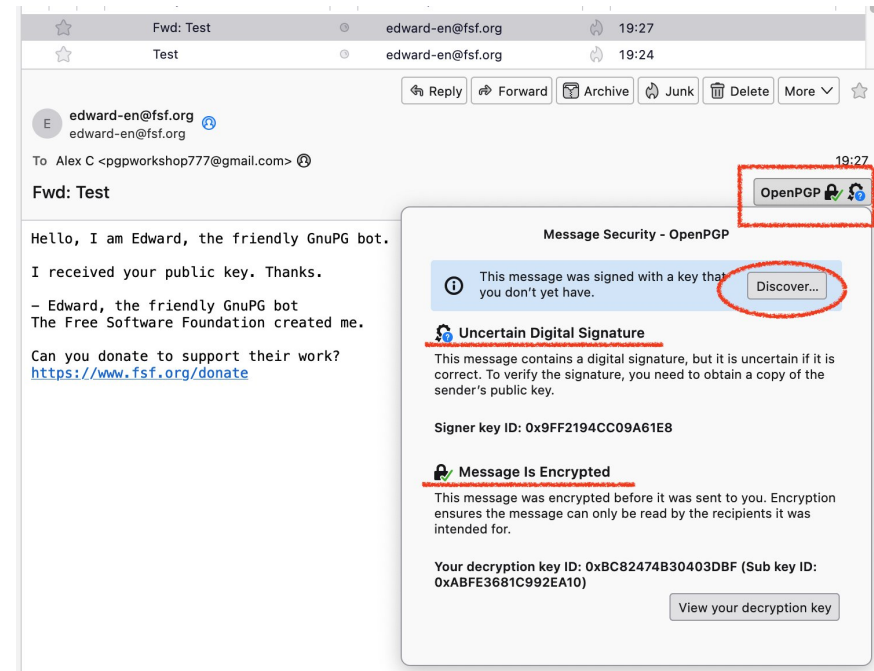
Free Software Foundation (FSF)
project to help start with OpenPGP

- Create email and attach Public Key
- Add one word to **Subject & Body**
- Sent to **edward@fsf.org**



Receive First Encrypted Email

- Congrats, you received Encrypted Email
- Edward has our Public Key and can encrypt message with our PubKey
- We haven't added Edward's PubKey so we are not sure if Edward is Edward
- We can Click Discover
- We initiated the email, we can Import and choose Import Accepted



Verify Keys

After we accepted the Keys,
They are accepted, but not Verified.
We can Verify the keys and
get green marks.

Edward, το φιλικό ρομπότ του GnuPG <edward-el@fsf.org>
Эдвард, дружелюбный GnuPG бот <edward-ru@fsf.org>

Type	public key
Key ID	0x9FF2194CC09A61E8
Fingerprint	F357 AA1A 5B1F A42C FD9F E52A 9FF2 194C C09A 61E8
Created	6/29/2014
Expiry	The key does not expire

Refresh Online

Your Acceptance Email addresses Certifications Structure

Do you accept this key for verifying digital signatures and for encrypting messages?

☐ No, reject this key.

☐ Not yet, maybe later.

☐ Yes, but I have not verified that it is the correct key.

☒ Yes, I've verified in person this key has the correct fingerprint.

Verify the fingerprint of the key using a secure communication channel other than email to make sure that it's really the key of edward-en@fsf.org.

Cancel **OK**

edward-en@fsf.org
edward-en@fsf.org

To: Alex C <pgpworkshop777@gmail.com>

Fwd: Test

Hello, I am Edward, the friendly GnuPG bot.

I received your public key. Thanks.

- Edward, the friendly GnuPG bot
The Free Software Foundation created me.

Can you donate to support their work?
<https://www.fsf.org/donate>

19:27

OpenPGP

Message Security - OpenPGP

Good Digital Signature

This message includes a valid digital signature from a key that you have already accepted. However, you have not yet verified that the key is really owned by the sender.

Signer key ID: 0x9FF2194CC09A61E8

View signer key

Message Is Encrypted

This message was encrypted before it was sent to you. Encryption ensures the message can only be read by the recipients it was

edward-en@fsf.org
edward-en@fsf.org

To: Alex C <pgpworkshop777@gmail.com>

Fwd: Test

Hello, I am Edward, the friendly GnuPG bot.

I received your public key. Thanks.

- Edward, the friendly GnuPG bot
The Free Software Foundation created me.

Can you donate to support their work?
<https://www.fsf.org/donate>

19:27

OpenPGP

Message Security - OpenPGP

Good Digital Signature

This message includes a valid digital signature from a verified key.

Signer key ID: 0x9FF2194CC09A61E8

View signer key

Message Is Encrypted

This message was encrypted before it was sent to you. Encryption ensures the message can only be read by the recipients it was intended for.

Your decryption key ID: 0xBC82474B30403DBF (Sub key ID: 0xABFE3681C992EA10)

View your decryption key

Discover PubKeys Online

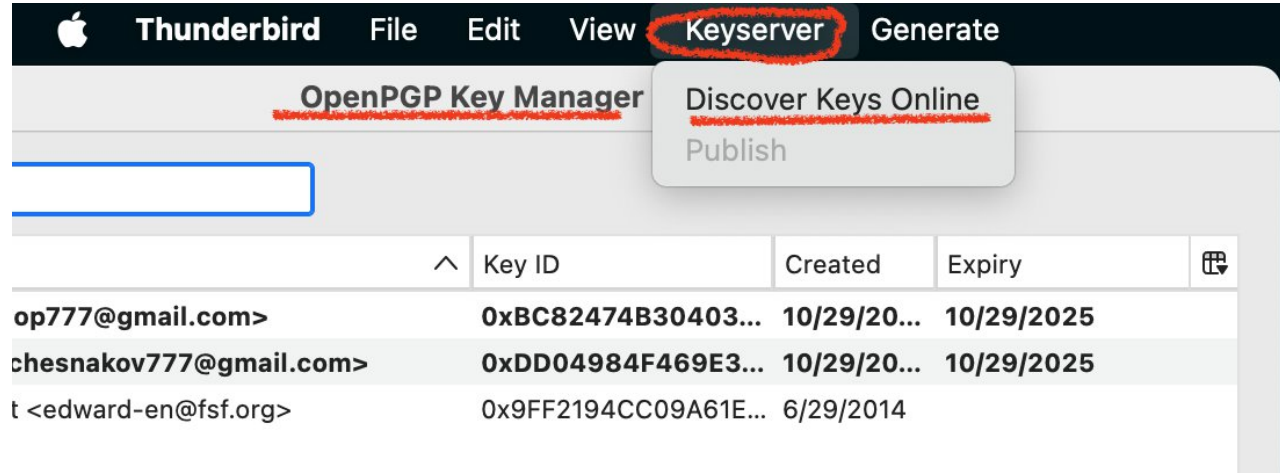
Thunderbird->

Tools->

OpenPGP Key Manager->

Keyserver->

Discover Keys Online

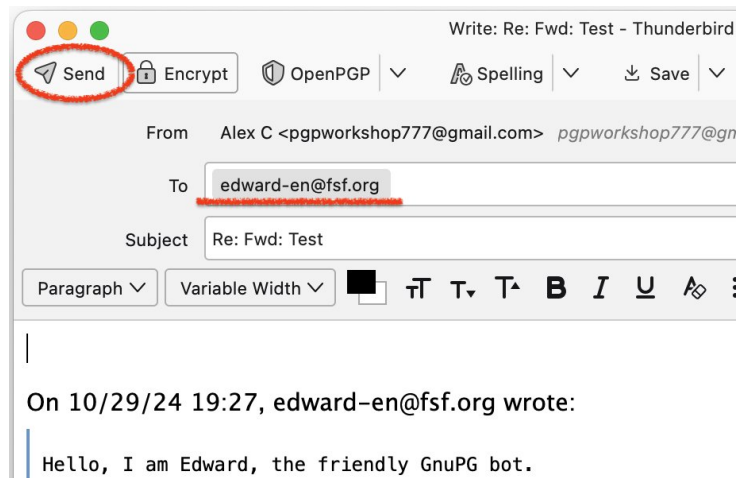
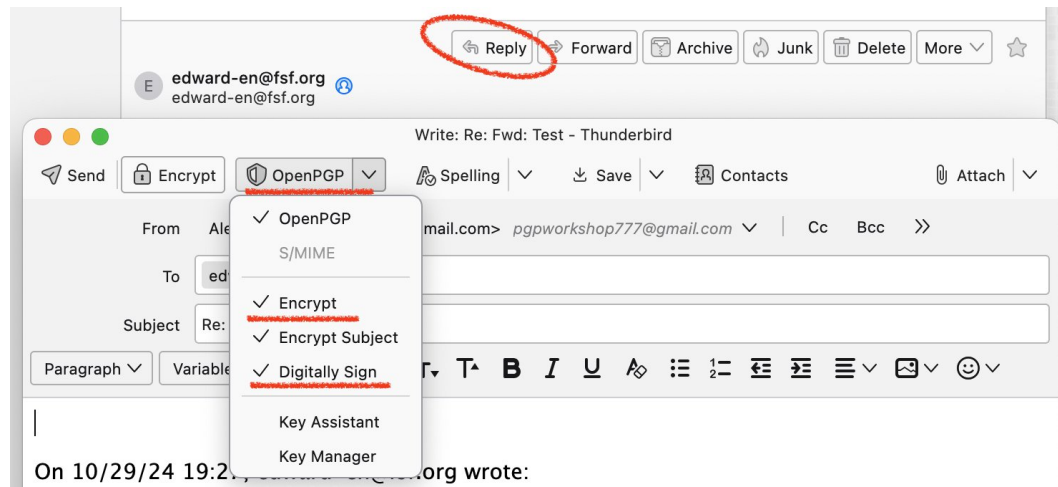


Try to find and Import

- edward-en@fsf.org
- pgpworkshop777@gmail.com

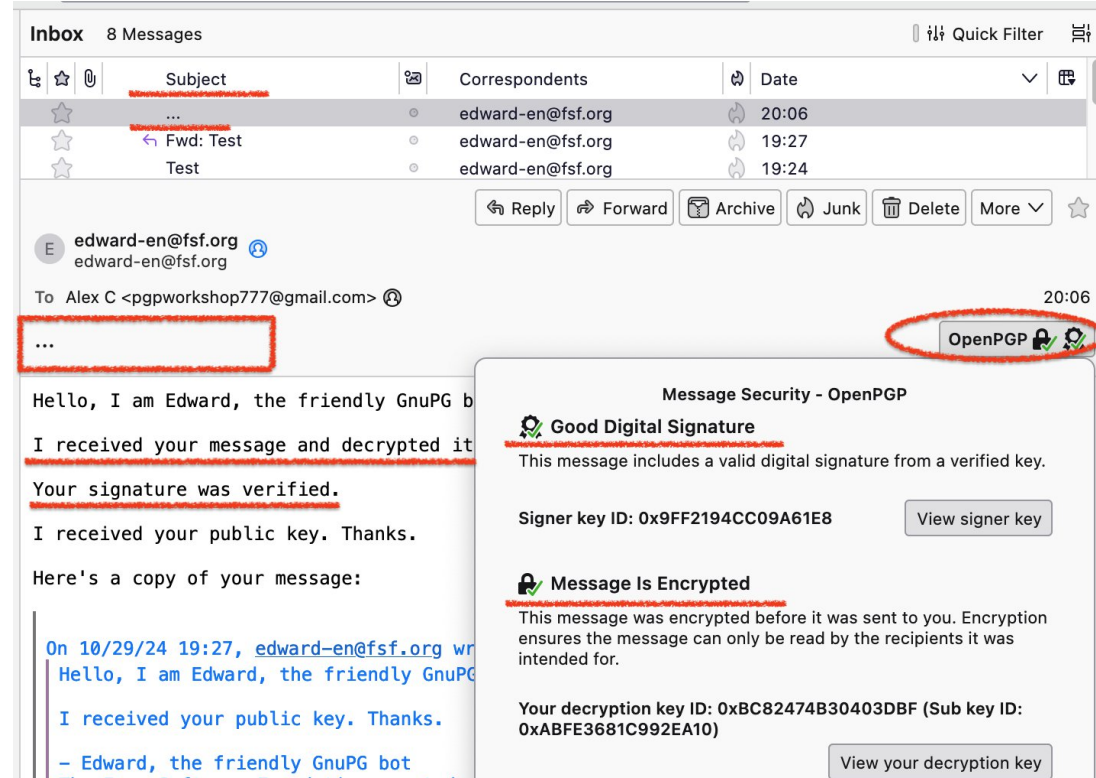
Send First Encrypted Email

- Reply to Edward and be sure to:
 - Encrypt
 - Digitally Sign
- Click Send



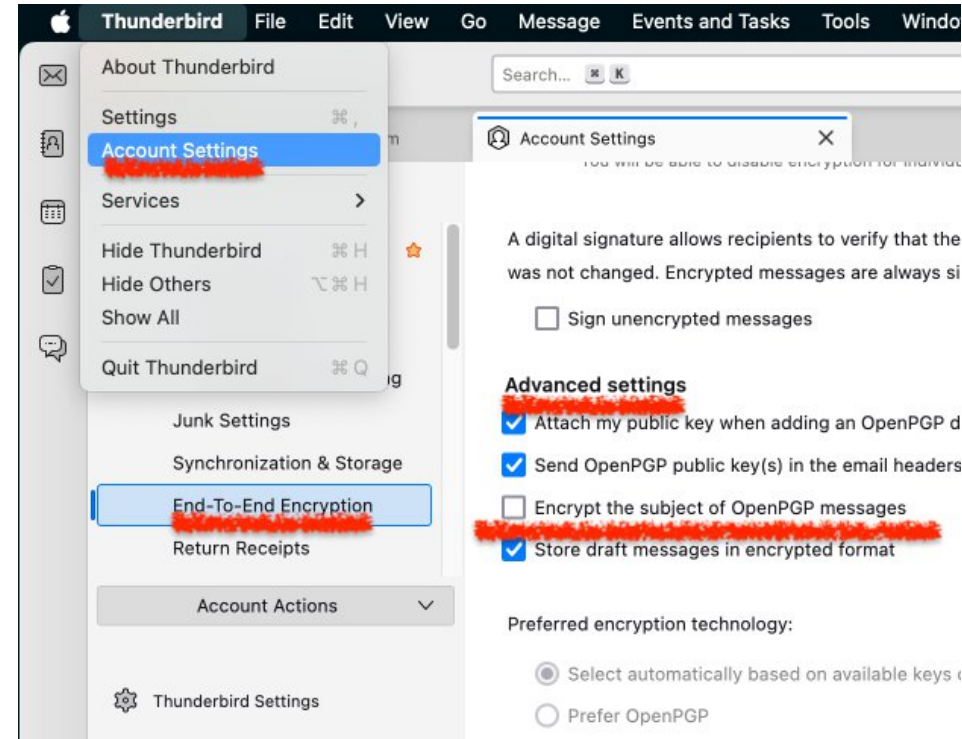
Receive an Encrypted Email from Edward

- Receive a response message
- Notice three dots instead of Subject



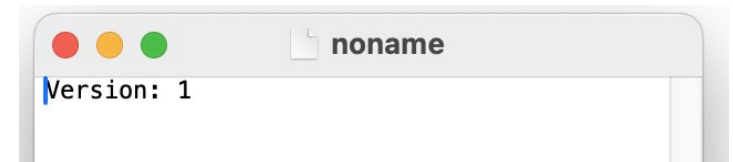
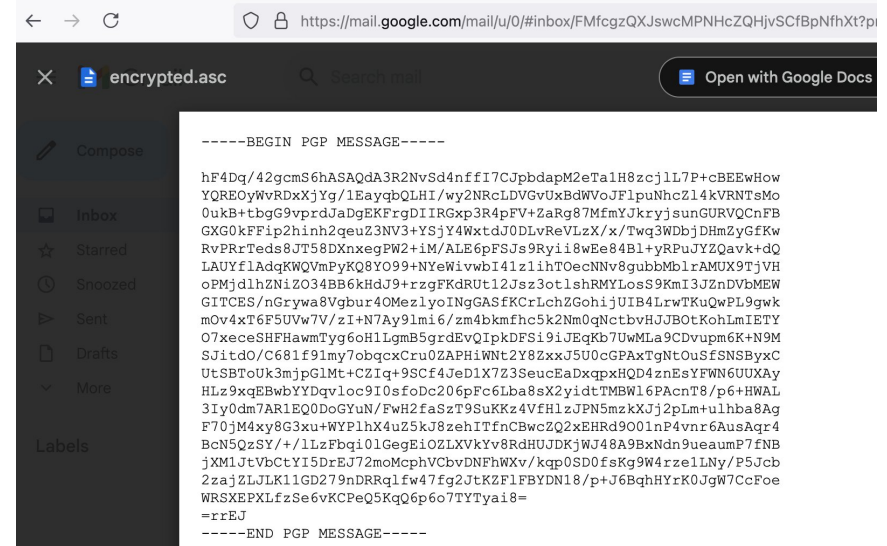
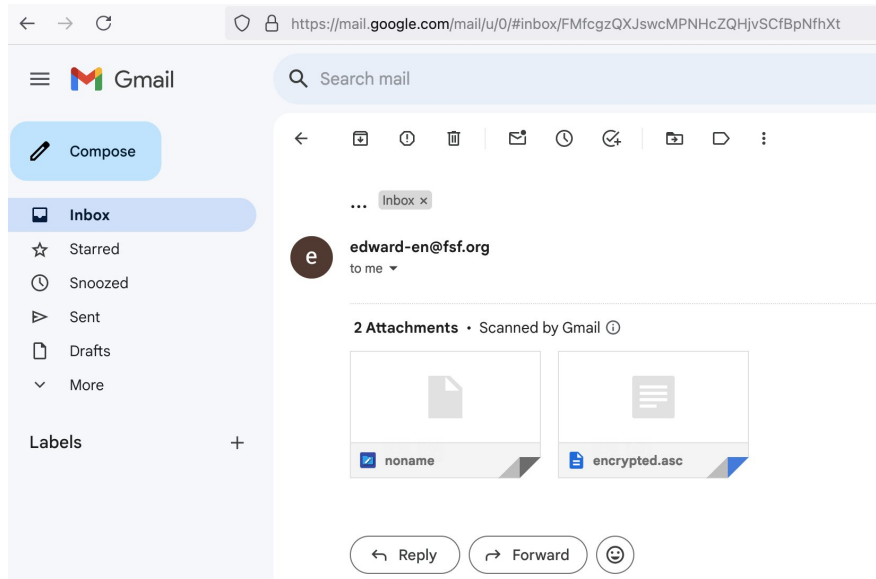
Subject Encryption

- OpenPGP do not offer Encryption for Subject
 - Thunderbird encrypts Subject by default
 - They use non-standard implementation
 - Works fine between Thunderbird clients
 - Do not work between everything else
 - Choose a neutral subject like “Hello” or “Test”
-
- **Do not encrypt subject**
 - Go to Account Settings->End-To-End Encryption->Advanced Settings->
 - Uncheck Encrypt Subject



Check if email is encrypted

- Log into email web portal
- See how the message looks
- Two attached files, no message body
 - encrypted.asc
 - noname



Why I can read the encrypted messages in the Sent folder?

We just discussed that message can only be decrypted with the matching Key.

If we encrypted message with the Edward's Public Key, it could be decrypted with the Edward's Private Key only, right?

Magic of OpenPGP – we can encrypt with multiple Public Key and decrypt with any of the matching Private Key

The screenshot shows an email interface. The email is from Alex C <pgpworkshop777@gmail.com> to edward-en@fsf.org, with the subject "Re: Fwd: Test". The message body contains a greeting, a thank you for the public key, and a donation link. A security panel on the right indicates a "Good Digital Signature" and that the "Message Is Encrypted". It provides the signer key ID (0xBC82474B30403DBF) and the decryption key ID (0xABFE3681C992EA10). Below the message, a table lists the keys used for encryption and decryption.

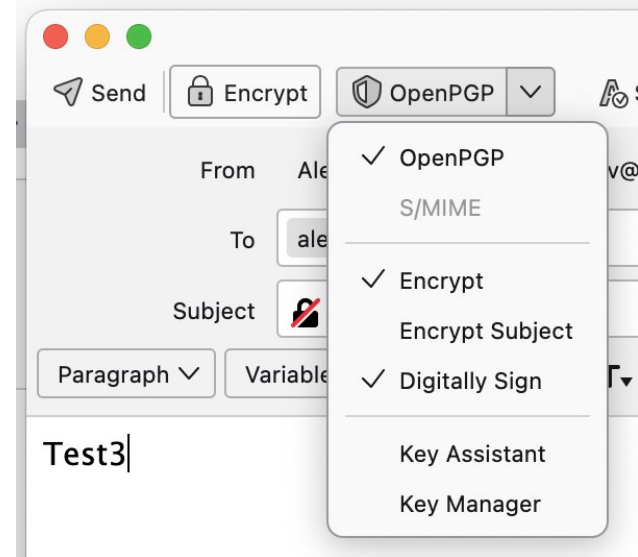
Email	Created	Fingerprint	Validity
<u>pgpworkshop777@gmail.com</u>	Oct 29, 2024	0D7B 2591 7AD7 AD57 07DD 92E8 BC82 474B 3040 3DBF	Valid
<u>achesnakov777@gmail.com</u>	Oct 29, 2024	01F4 174A 511C 84A1 8678 FA23 DD04 984F 469E 34AD	Valid
<u>edward-en@fsf.org</u>	Jun 29, 2014	F357 AA1A 5B1F A42C FD9F E52A 9FF2 194C C09A 61E8	Valid
<u>veracrypt@idrix.fr</u>	Sep 11, 2018	5069 A233 D55A 0EEB 174A 5FC3 821A CD02 680D 16DE	Valid

Exchange encrypted email with a person

- Ask somebody around for their email that has OpenPGP keys
- As a backup, you can send a email to me at **pgpworkshop777@gmail.com**
- Search for their keys at Thunderbird's OpenPGP Key Manager or at <https://keys.openpgp.org/>
- Import keys
- Click on the keys and Accept keys with "Yes, I've verified in person..."
- Create a new email to this person
- Be sure you Encrypt and Digitally sign the message

Other person:

- Receive the email
- Import Keys
- Click Reply to the email
- Encrypt and Digitally Sign the email before Sending



Advanced use of GnuPG (GPG)



GPGTools

Support Plan

GPG Suite

One simple package
with everything you need,
to protect your emails and files.

Download

for macOS 10.15 - 13.x

By downloading, you agree to our [Terms of Distribution](#)

Includes a 30-day trial of GPG Mail. For continued
use of GPG Mail, please purchase a [support plan](#)

Gpg4win Components

Gpg4win is an installer for Windows and contains several Free Software components:



GnuPG

The backend; this is the actual encryption tool.



Kleopatra

A certificate manager for OpenPGP and X.509 (S/MIME) and common
crypto dialogs.



GpgOL

An add-in for Microsoft Outlook providing email cryptography. Supports
both IMAP/SMTP and MS Exchange servers.



GpgEX

A plugin for Microsoft Explorer (file encryption).

Install GnuPG (CLI) and GUI tools

GnuPG - <https://gnupg.org/>

Mac. GPG Suite. Includes CLI and GUI:

```
brew install --cask gpg-suite
```

Kali Linux. GnuPG and Kleopatra (gpg is pre-installed):

```
sudo apt update
```

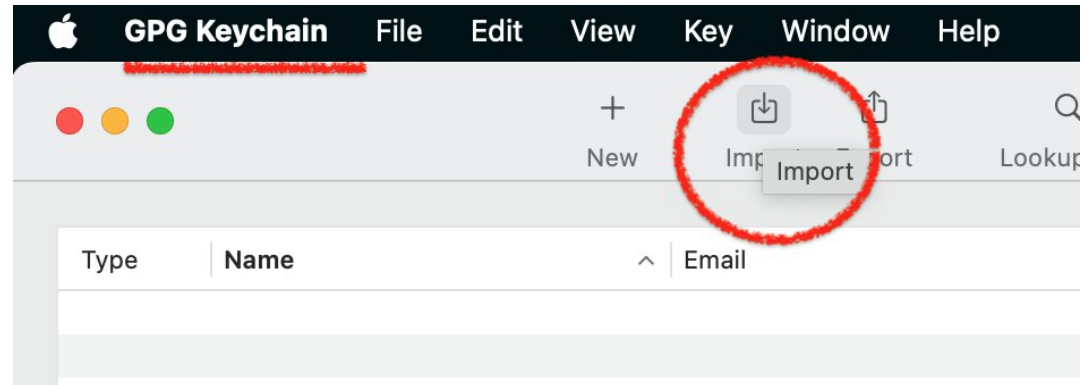
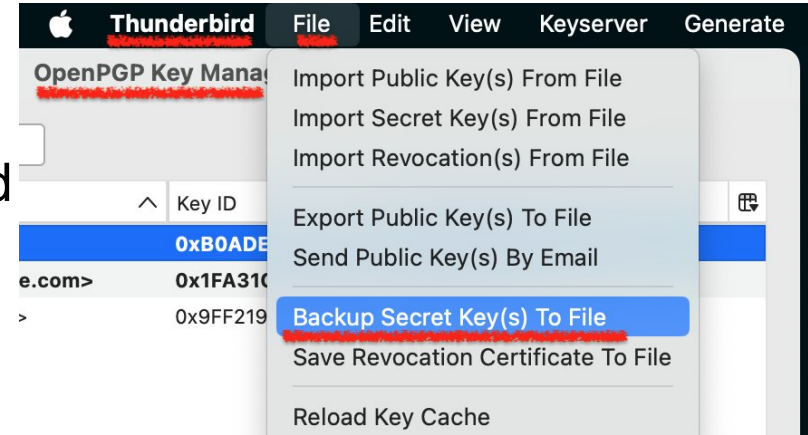
```
sudo apt install gpg kleopatra
```

Windows. Gpg4win. Includes GnuPG and Kleopatra:

<https://gpg4win.org/> Click Download. Install

Export Keys from Thunderbird to GnuPG

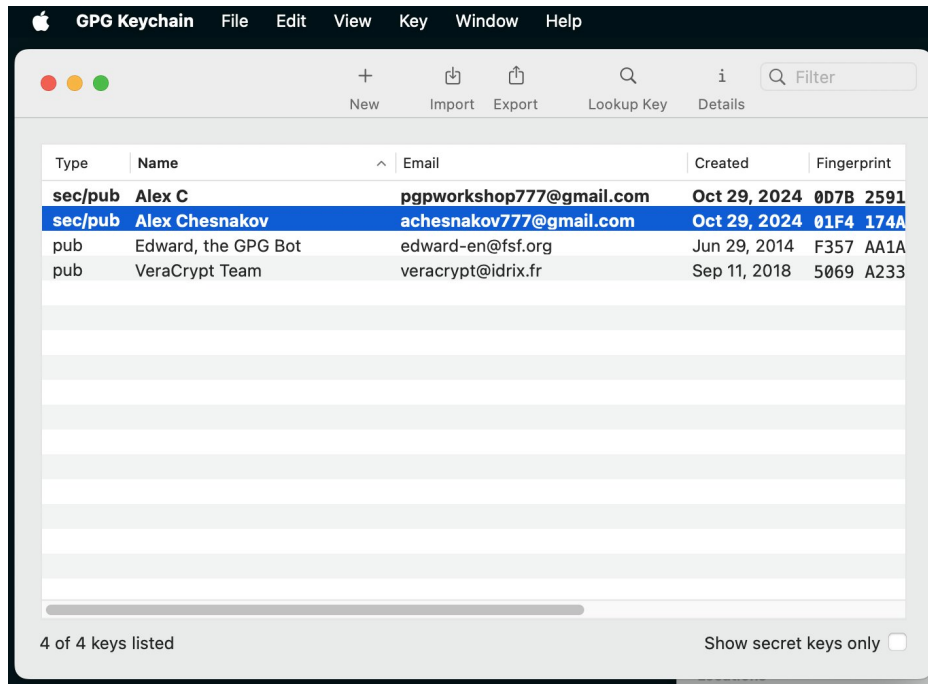
- Thunderbird can't sync keys with GnuPG
 - Export Secret (Private) Keys from Thunderbird
 - Thunderbird will export Private and Pub Keys
 - Choose Password for Private (Secret) Keys
-
- Import keys to GPG Keychain or Kleopatra



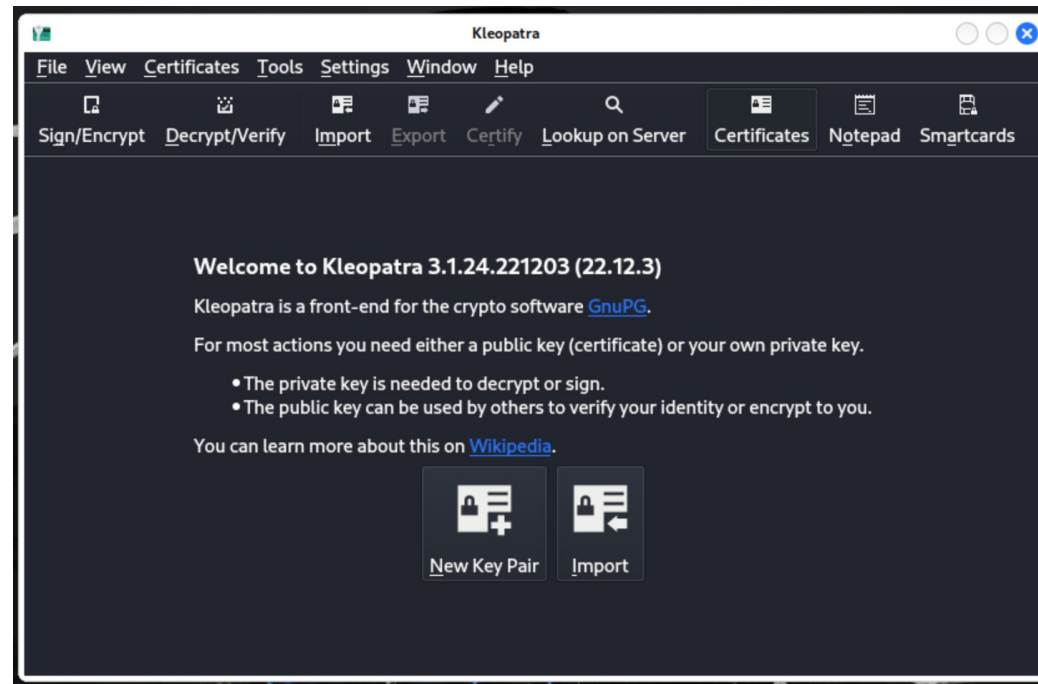
CLI vs GUI

- CLI version offers full functionality
- For simple use both CLI and GUI are good
- CLI version can be used in scripts and automation

macOS



Linux / Windows



List Keys

- **gpg --list-keys** – List Public Keys
- **gpg --list-secret-keys** – List Private Keys

```
22:01 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --list-keys
/Users/alexc/.gnupg/pubring.kbx
-----
pub   rsa2048 2014-06-29 [SC]
      F357AA1A5B1FA42CFD9FE52A9FF2194CC09A61E8
uid     [ unknown] Edward, the GPG Bot <edward-en@fsf.org>
uid     [ unknown] Edward the GPG Bot <edward@fsf.org>
uid     [ unknown] GnuPGボットのEdward <edward-ja@fsf.org>
uid     [ unknown] Edward, l'amichevole bot GnuPG <edward-it@fsf.org>
uid     [ unknown] Edward, le gentil robot de GnuPG <edward-fr@fsf.org>
uid     [ unknown] Edward, el simpático robot GnuPG <edward-es@fsf.org>
uid     [ unknown] Edward, o amigo robô de GnuPG <edward-pt-br@fsf.org>
uid     [ unknown] Edward, robotul GnuPG cel prietenos <edward-ro@fsf.org>
uid     [ unknown] Edward, arkadaş canlısı GnuPG botu <edward-tr@fsf.org>
uid     [ unknown] Edward, der freundliche GnuPG Roboter <edward-de@fsf.org>
uid     [ unknown] Эдвард, дружелюбный GnuPG бот <edward-ru@fsf.org>
uid     [ unknown] Edward, το φιλικό ρομπότ του GnuPG <edward-el@fsf.org>
uid     [ unknown] 爱德华, 一个友好的 GnuPG 机器人 <edward-zh@fsf.org>
sub   rsa2048 2014-06-29 [E]

pub   rsa4096 2018-09-11 [SC]
      5069A233D55A0EEB174A5FC3821ACD02680D16DE
uid     [ unknown] VeraCrypt Team (2018 - Supersedes Key ID=0x54DD0393) <veracrypt@idrix.fr>
sub   rsa4096 2018-09-11 [E]
sub   rsa4096 2018-09-11 [A]

pub   ed25519 2024-10-30 [SC] [expires: 2025-10-30]
      0D7B25917AD7AD5707DD92E8BC82474B30403DBF
uid     [ultimate] Alex C <pgpworkshop777@gmail.com>
sub   cv25519 2024-10-30 [E] [expires: 2025-10-30]

pub   ed25519 2024-10-30 [SC] [expires: 2025-10-30]
      01F4174A511C84A18678FA23DD04984F469E34AD
uid     [ultimate] Alex Chesnakov <achesnakov777@gmail.com>
sub   cv25519 2024-10-30 [E] [expires: 2025-10-30]
```

```
22:02 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --list-secret-keys
/Users/alexc/.gnupg/pubring.kbx
-----
sec   ed25519 2024-10-30 [SC] [expires: 2025-10-30]
      0D7B25917AD7AD5707DD92E8BC82474B30403DBF
uid     [ultimate] Alex C <pgpworkshop777@gmail.com>
ssb   cv25519 2024-10-30 [E] [expires: 2025-10-30]

sec   ed25519 2024-10-30 [SC] [expires: 2025-10-30]
      01F4174A511C84A18678FA23DD04984F469E34AD
uid     [ultimate] Alex Chesnakov <achesnakov777@gmail.com>
ssb   cv25519 2024-10-30 [E] [expires: 2025-10-30]
```

Export Keys

```
gpg --export-secret-keys --armor [keyID] > my_pgpkey.private.asc
```

```
gpg --export --armor [keyID] > my_pgpkey.public.asc
```

[keyID] – **Last 8 or 16** digits of Fingerprint
may use Email Address
 or Full Fingerprint

--armor – Export as ASCII text

```
>20:36 alexc@Alexs-MBP /Users/alexc
% gpg --list-keys edward-en@fsf.org
pub  rsa2048 2014-06-29 [SC]
    [F357AA1A5B1FA42CFD9FE52A9FF2194CC09A61E8 ]
uid   [ unknown] Edward, the GPG Bot <edward-en@fsf.org>
uid   [ unknown] Edward the GPG Bot <edward@fsf.org>
uid   [ unknown] GnuPGボットのEdward <edward-ja@fsf.org>
uid   [ unknown] Edward, l'amichevole bot GnuPG <edward-it@fsf.org>
uid   [ unknown] Edward, le gentil robot de GnuPG <edward-fr@fsf.org>
uid   [ unknown] Edward, el simpático robot GnuPG <edward-es@fsf.org>
uid   [ unknown] Edward, o amigo robô de GnuPG <edward-pt-br@fsf.org>
uid   [ unknown] Edward, robotul GnuPG cel prietenos <edward-ro@fsf.org>
uid   [ unknown] Edward, arkadaş canlısı GnuPG botu <edward-tr@fsf.org>
uid   [ unknown] Edward, der freundliche GnuPG Roboter <edward-de@fsf.org>
uid   [ unknown] Эдвард, дружелюбный GnuPG бот <edward-ru@fsf.org>
uid   [ unknown] Edward, το φιλικό ρομπότ του GnuPG <edward-el@fsf.org>
uid   [ unknown] 爱德华, 一个友好的 GnuPG 机器人 <edward-zh@fsf.org>
sub   rsa2048 2014-06-29 [E]

>20:36 alexc@Alexs-MBP /Users/alexc
% gpg --export --armor C09A61E8 > edward-en_fsf.org.public.asc
>20:36 alexc@Alexs-MBP /Users/alexc
% gpg --export --armor edward-en@fsf.org > edward-en_fsf.org.public.asc
>20:37 alexc@Alexs-MBP /Users/alexc
% gpg --export --armor F357AA1A5B1FA42CFD9FE52A9FF2194CC09A61E8 > edward-en_fsf.org.public.asc
>20:38 alexc@Alexs-MBP /Users/alexc
% cat edward-en_fsf.org.public.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQENBF0wfzoBCADpwK6sGC3EUzgD7IW1X5ZDR1nC5/rcXacAJLarPpvQBEz4pwjT
joAzATM7F9RwIzJ3hJTZhYaQY4cfiGLKSnr8GPC8A4QkxXIaQ0hLpcsBSbtZJp
o2i0zL2FRHmW2Z1nSHXPKbDwx5p0NcdQfjL9i2Yo31aLI0/Chhn5uyvIzn0jaCSC
/O6x2C4m81Lu+B4UTDpl8y6ChtphUxyFGd7RXDXmkYQrxVqJbXKuSVmNmH09myG
Zi0110Y10cCxa3TYDn0ktp49BbMGR9w14eDTE8GHEzRjmt4bFphTQW/1N4Yk0UV
```

Delete Keys

```
gpg --delete-secret-keys [keyID]
```

```
gpg --delete-keys [keyID]
```

[keyID] – **Last 8 or 16** digits of Fingerprint
may use Email Address
or Full Fingerprint

```
22:10 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --delete-keys edward-en@fsf.org
gpg (GnuPG/MacGPG2) 2.2.41; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

pub  rsa2048/9FF2194CC09A61E8 2014-06-29 Edward, the GPG Bot <edward-en@fsf.org>
Delete this key from the keyring? (y/N) y
22:11 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --list-keys
/Users/alexc/.gnupg/pubring.kbx
-----
pub  rsa4096 2018-09-11 [SC]
     5069A233D55A0EEB174A5FC3821ACD02680D16DE
uid          [ unknown] VeraCrypt Team (2018 - Supersedes Key ID=0x54DDD393) <veracrypt@idrix.fr>
sub  rsa4096 2018-09-11 [E]
sub  rsa4096 2018-09-11 [A]

pub  ed25519 2024-10-30 [SC] [expires: 2025-10-30]
     0D7B25917AD7AD5707DD92E8BC82474B30403DBF
uid          [ultimate] Alex C <pgpworkshop777@gmail.com>
sub  cv25519 2024-10-30 [E] [expires: 2025-10-30]

pub  ed25519 2024-10-30 [SC] [expires: 2025-10-30]
     01F4174A511C84A18678FA23DD04984F469E34AD
uid          [ultimate] Alex Chesnakov <achesnakov777@gmail.com>
sub  cv25519 2024-10-30 [E] [expires: 2025-10-30]
22:11 alexc@Alexs-MacBook-Pro /Users/alexc
```

Import Keys

```
gpg --import my_pgpkey.private.asc  
gpg --import my_pgpkey.public.asc
```

```
22:13 alexc@Alexs-MacBook-Pro /Users/alexc  
% gpg --import edward-en_fsf.org.public.asc  
gpg: key 9FF2194CC09A61E8: public key "Edward, the GPG Bot <edward-en@fsf.org>" imported  
gpg: Total number processed: 1  
gpg: imported: 1
```

Keyserver

```
gpg --keyserver hkps://keys.openpgp.org --search edward-en@fsf.org
```

We can **skip** `--keyserver hkps://keys.openpgp.org`
if we are using **default** `keys.openpgp.org` server,
not custom, like `pgp.mit.edu`

```
gpg --search pgpworkshop777@gmail.com
```

Email address may have multiple Keys,
need to specify specific Key

```
gpg --recv-keys BC82474B30403DBF
```

```
gpg --send-keys BC82474B30403DBF
```

```
19:27 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --search pgpworkshop777@gmail.com
gpg: data source: https://keys.openpgp.org:443
(1) Alex C <pgpworkshop777@gmail.com>
    256 bit EDDSA key BC82474B30403DBF, created: 2024-10-30
Keys 1-1 of 1 for "pgpworkshop777@gmail.com". Enter number(s), N)ext, or Q)uit > 1
gpg: key BC82474B30403DBF: "Alex C <pgpworkshop777@gmail.com>" not changed
gpg: Total number processed: 1
gpg: unchanged: 1
19:27 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --recv-keys BC82474B30403DBF
gpg: key BC82474B30403DBF: "Alex C <pgpworkshop777@gmail.com>" not changed
gpg: Total number processed: 1
gpg: unchanged: 1
19:28 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --send-keys BC82474B30403DBF
gpg: sending key BC82474B30403DBF to hkps://keys.openpgp.org
```


Generate Keys

gpg --full-generate-key

```
% gpg --expert --full-generate-key
gpg (GnuPG/MacGPG2) 2.2.41; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

```
Please select what kind of key you want:
(1) RSA and RSA (default)
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(7) DSA (set your own capabilities)
(8) RSA (set your own capabilities)
(9) ECC and ECC
(10) ECC (sign only)
```

Revocation Certificate:

- Save it in a safe place
- Upload to keyserve
- if you lost your Private Key
- Decryption will still work
- after you upload Rev.Cert.
- Somebody need to search
- keyserver to notice that
- the key was revoked

```
19:36 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --full-generate-key
gpg (GnuPG/MacGPG2) 2.2.41; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

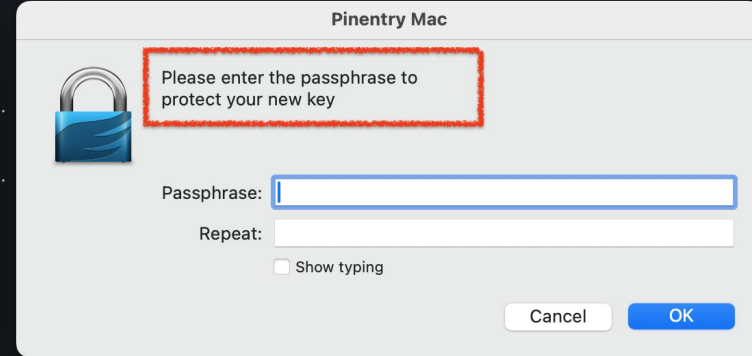
```
Please select what kind of key you want:
(1) RSA and RSA (default)
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072)
Requested keysize is 3072 bits
Please specify how long the key should be valid.
  0 = key does not expire
<n> = key expires in n days
<n>w = key expires in n weeks
<n>m = key expires in n months
<n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Thu Oct 30 19:36:59 2025 PDT
Is this correct? (y/N) y
```

GnuPG needs to construct a user ID to identify your key.

```
Real name: Alex Chesnakov
Email address: achesnakov777@gmail.com
Comment:
You selected this USER-ID:
"Alex Chesnakov <achesnakov777@gmail.com>"
```

```
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? 0
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/Users/alexc/.gnupg/openpgp-revocs.d/DB4390CB194FD2C74516460625174BFF5A1DD5CE.rev'
public and secret key created and signed.
```

```
pub  rsa3072 2024-10-31 [SC] [expires: 2025-10-31]
     DB4390CB194FD2C74516460625174BFF5A1DD5CE
uid                               Alex Chesnakov <achesnakov777@gmail.com>
sub  rsa3072 2024-10-31 [E] [expires: 2025-10-31]
```



Sign the message / file

`gpg --sign file`

output: file.**gpg** (input file + signature, binary)

`gpg --clear-sign file`

output: file.**asc** (original message + ascii signature)

`gpg --detach-sign file`

output: file.**sig** (separate signature, binary)

`gpg --armor --local-user 30403DBF --detach-sig file`

output: file.**asc** (specify private key, separate signature, ascii)

```
20:05 alexc@Alexs-MacBook-Pro /Users/alexc
% gpg --armor --local-user 30403DBF --detach-sig my_message.txt
20:06 alexc@Alexs-MacBook-Pro /Users/alexc
% cat my_message.txt.asc
-----BEGIN PGP SIGNATURE-----

iHUEABYIAB0WIIQQNeyWRtetVwfdkui8gkdLMEA9vwUCZyL0PAAKCRc8gkdLMEA9
v0BhAQcNxXVE0Cvc34qBAWH/bdM7TEHmhz9dT0W+78PX5hSvFQD9H1m3xu0+2ffK
xlu3GVTPEBgmWqChXqDPyw6ap2wxqgw=
=w0Tv
-----END PGP SIGNATURE-----
```

```
% cat my_message.txt
This is my message.
22:31 alexc@Alexs-MBP /Users/alexc
% gpg --sign my_message.txt
22:31 alexc@Alexs-MBP /Users/alexc
% cat my_message.txt.gpg
000000!0a0000u000H0~00M-.NL00+0(I;000Y0D00
000R1Y1E0*50 0+0000000000)i~000sq00000b0g60
000Wr0M<0EcX0f3#â<;5
^0p060U0010%6~00~00g/3%
```

```
% gpg --clear-sign my_message.txt
22:33 alexc@Alexs-MBP /Users/alexc
% cat my_message.txt.asc
-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA256

This is my message.
-----BEGIN PGP SIGNATURE-----

iHUEARYIAB0WIIQQJeiY6kL95Fs/EEBSwreqgg+e+iAUCZsQqugAKCRCwreqgg+e+
iFdSAQCSCMcDIIXEUYCYAmK16/mS/HzKNiXP+pSZKPUKSY1SnyQD9G/W88/STb00d
8qU94jTwxsMtkr0xHYyvpqiNw6AHwU=
=NyDr
-----END PGP SIGNATURE-----
```

```
% gpg --detach-sign my_message.txt
22:48 alexc@Alexs-MBP /Users/alexc
% cat my_message.txt.sig
0! z&:00y000000f0.7
00000=00*00:00 S400r900Z#0h0080}W0000`00000g0VwKP#000IM{y00H0N0%
```


Verify Signature

```
gpg --verify my_message.txt.sig
```

```
20.11 alexc@ALEX:~$ gpg --verify my_message.txt.asc  
% gpg --verify my_message.txt.asc  
gpg: assuming signed data in 'my_message.txt'  
gpg: Signature made Wed Oct 30 20:06:36 2024 PDT  
gpg:                using EDDSA key 0D7B25917AD7AD5707DD92E8BC82474B30403DBF  
gpg: Good signature from "Alex C <pgpworkshop777@gmail.com>" [ultimate]
```

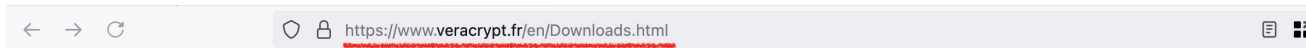
Verifying Files from the Internet, Veracrypt

Go to <https://www.veracrypt.fr/en/Downloads.html>

Download

- README.TXT
- README.TXT.sig

Notice PGP Keys
ID=0x680D16DE



- *Full list of changes:* [README.txt](#) ([PGP Signature](#))
- *Checksums:*
 - [SHA256 sums for VeraCrypt 1.26.7 release files](#) ([PGP Signature](#))
 - [SHA512 sums for VeraCrypt 1.26.7 release files](#) ([PGP Signature](#))
- *Bundle containing all release files:*
 - [VeraCrypt 1.26.7 Bundle.7z](#) ([PGP Signature](#))
 - [SHA256/SHA512 sums of VeraCrypt 1.26.7 Bundle](#) ([PGP Signature](#))

Previous Versions Archives

The packages and installers of all previous VeraCrypt versions can be found at <https://sourceforge.net/projects/veracrypt/files/>.

Important Notice for VeraCrypt RPM Packages

VeraCrypt signs its RPM packages using the standard RPM signature mechanism which embeds the signature into the packages. This makes verification easier for users. Follow these steps:

1. **Import VeraCrypt GPG Public Key:** In a terminal window, use the following command to import the VeraCrypt GPG public key:

```
sudo rpm --import https://www.idrix.fr/VeraCrypt/VeraCrypt_PGP_public_key.asc
```

This process will display the key's fingerprint and ask for your confirmation before adding the key.

2. **Validate the Key's Fingerprint:** During the key import process, carefully verify that the displayed fingerprint matches the official VeraCrypt fingerprint:

```
5069 A233 D55A 0EEB 174A 5FC3 821A CD02 680D 16DE
```

This step is vital for your security. Do not proceed if the fingerprints do not match.

🔒 By following these steps, you help ensure the integrity of the software you install, safeguarding your system from potential threats.

Remember, your security is paramount. Always verify the authenticity of your downloads.

[What are PGP signatures?](#)

PGP Public Key: https://www.idrix.fr/VeraCrypt/VeraCrypt_PGP_public_key.asc (ID=0x680D16DE, Fingerprint=5069A233D55A0EEB174A5FC3821ACD02680D16DE)

Verifying Files from the Internet, Veracrypt (Continue)

```
gpg --verify README.TXT.sig README.TXT
```

You can `--import` Key and **trust** this Key to avoid WARNING

```
20:22 alexc@Alexs-MacBook-Pro ~/Users/alexc
% gpg --verify README.TXT.sig README.TXT
gpg: Signature made Sun Aug 25 02:29:07 2024 PDT
gpg: using RSA key 5069A233D55A0EEB174A5FC3821ACD02680D16DE
gpg: key 821ACD02680D16DE: public key "VeraCrypt Team (2018 - Supersedes Key ID=0x54DDD393) <veracrypt@idrix.fr>" imported
gpg: Total number processed: 1
gpg: imported: 1
gpg: Good signature from "VeraCrypt Team (2018 - Supersedes Key ID=0x54DDD393) <veracrypt@idrix.fr>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg: There is no indication that the signature belongs to the owner.
Primary key fingerprint: 5069 A233 D55A 0EEB 174A 5FC3 821A CD02 680D 16DE
20:23 alexc@Alexs-MacBook-Pro ~/Users/alexc
```

Encrypt / Decrypt Files

```
gpg -e -s -a -u pgpworkshop777@gmail.com -r achesnakov777@gmail.com message.txt
```

```
gpg -d message.txt.asc
```

-e, --encrypt

-s, --sign

-a, --armor

-d, --decrypt

-u, --local-user USER-ID
use USER-ID to sign, encr, decr

-r, --recipient USER-ID
encrypt for USER-ID

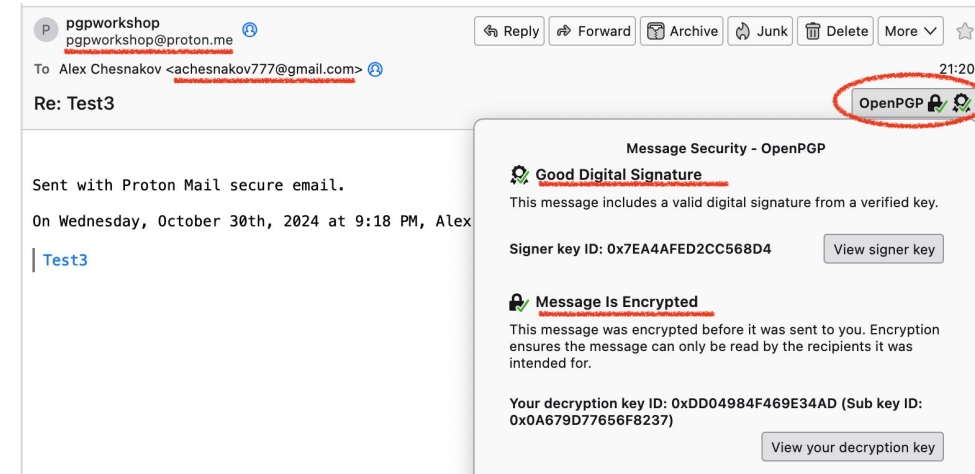
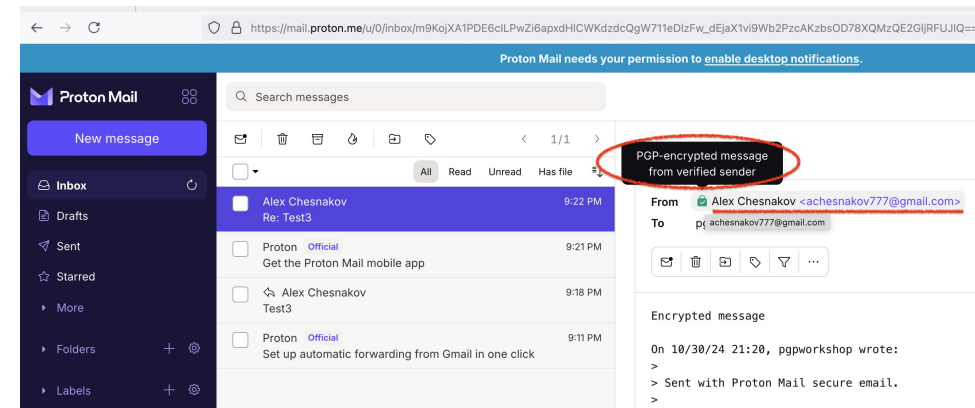
Note: You can add multiple **recipients**, including yourself to be able to decrypt the message

```
alex@alex-MBP /Users/alex
% gpg -e -s -a -u achesnakov@mailfence.com -r achesnakov@mailfence.com message.txt
22:30 alex@Alexs-MBP /Users/alex
% cat message.txt
This is my message.
22:30 alex@Alexs-MBP /Users/alex
% gpg --decrypt message.txt.asc
gpg: encrypted with 256-bit ECDH key, ID 1492A775A41C110D, created 2024-08-16
      "Alex Chesnakov <achesnakov@mailfence.com>"
This is my message.
gpg: Signature made Tue Aug 20 22:30:16 2024 PDT
gpg:       using EDDSA key 4AA6B476861C007DDE95FC2B1FA31C2070C6150F
gpg:       issuer "achesnakov@mailfence.com"
gpg: Good signature from "Alex Chesnakov <achesnakov@mailfence.com>" [ultimate]
22:30 alex@Alexs-MBP /Users/alex
% cat message.txt.asc
-----BEGIN PGP MESSAGE-----

hF4DFJKndaQcEQ0SAQdAaRSq0EFYSeh0+jqcTh0C31ntDgLLilFfZtZHfPyLGhEw
l4ruMI/KwC9orEv7aHrWmDnewbLayMFQu4juicNQRSbnNKThpEhzxUvUJoCc7qgJ
0sAoAU0xsWSWESjl5bi/Pu0/iWx/aWZmTQvtFWjV10jA0q7yP3hGEck1STHmBpFK
wldmBA1MLVFB1L5kVvQkwsWNZTvpYUbvLkLchERYZfSI7f8R1m+PX8ChSxErt5z
X0bKHnUMja+09vtctXwyS83kaj2ML/eYV3hZ7FazFHUozYiEZzfX0KI0h42o8w7h
MA0LV39ce72AzfwIzsTaGLqateIy4BPslxAVeM11j6qppsokfc+Q5ioI9ERaxrzc
tCLS4Ey8Jj03I3hgBGfMKAQc6okyAfAvsWCT4c4356g5Ly0vvNynTDqcpq==
=qD4q
-----END PGP MESSAGE-----
```

Proton Mail and mobile clients

- Proton Mail is OpenPGP compatible and works with non-Proton email
- Proton Mail is handling encryption
- You will not be able to use Thunderbird
- Proton Mail Bridge is a way to use std. email client, but it handles encryption
- Proton Mail clients are available for macOS, Windows, Linux, iOS, Android
- Thunderbird Mobile for Android



Not Covered

- Web of Trust
- Traditional OpenPGP keyservers
- Key Signing Parties
- Photo IDs in OpenPGP keys
- Sub-Keys
- Edit Keys (`gpg --edit-key`)
- Physical Keys: YubiKeys, Smartcards

Q&A

- SSL/TLS and SSH use public / private keys as well. What the difference?

Resources:

- <https://emailselfdefense.fsf.org/en/>
- <https://www.gnupg.org/gph/en/manual.html>
- <https://www.oreilly.com/library/view/pgp-gpg/9781457185212/>
Free O'REILLY access with San Jose Public Library
<https://www.sjpl.org/resource/oreilly/>

Thanks



https://github.com/olexandrch/OpenPGP_Email_Encryption_Workshop



<https://www.linkedin.com/in/alex-chesnakov/>



Alex Chesnakov

